```
     # ./a_spec.rb:13:in `block (2 levels) in <top (required)>'

  has an example that succeeds

Finished in 0.03249 seconds (files took 0.08034 seconds to load)
4 examples, 2 failures

Failed examples:

rspec ./a_spec.rb:2 # A group with a failure has an example that fails
rspec ./a_spec.rb:12 # Another group with a failure has an example that fails
```

As before, we're seeing failure messages interspersed with the rest of the output, instead of gathered at the end. Now that you've configured RSpec to use this formatter, and have seen it in action, let's walk through how it actually works.

How Formatters Work

A formatter goes through three main steps:

1. Register itself with RSpec to receive specific notifications
2. Initialize itself at the beginning of the RSpec run
3. React to events as they occur

As we talk about those steps in detail, you may want to look at the formatter's source code. If you open rspec-print_failures_eagerly/lib/rspec/print_failures_eagerly.rb from your home directory, you'll see the following Ruby class:

09-configuring-rspec/02/configuring_rspec/rspec/print_failures_eagerly.rb

```
Line 1  module RSpec
        module PrintFailuresEagerly
          class Formatter
            RSpec::Core::Formatters.register self, :example_failed
        5
            def initialize(output)
              @output = output
              @last_failure_index = 0
            end
       10
            def example_failed(notification)
              @output.puts
              @output.puts notification.fully_formatted(@last_failure_index += 1)
              @output.puts
            end
       15   end
        end
      end
```